

Solving problems by searching

Chapter 3

Outline

- Problem-solving agents
- Problem types
- Problem formulation
- Example problems
- Basic search algorithms

Problem-solving agents

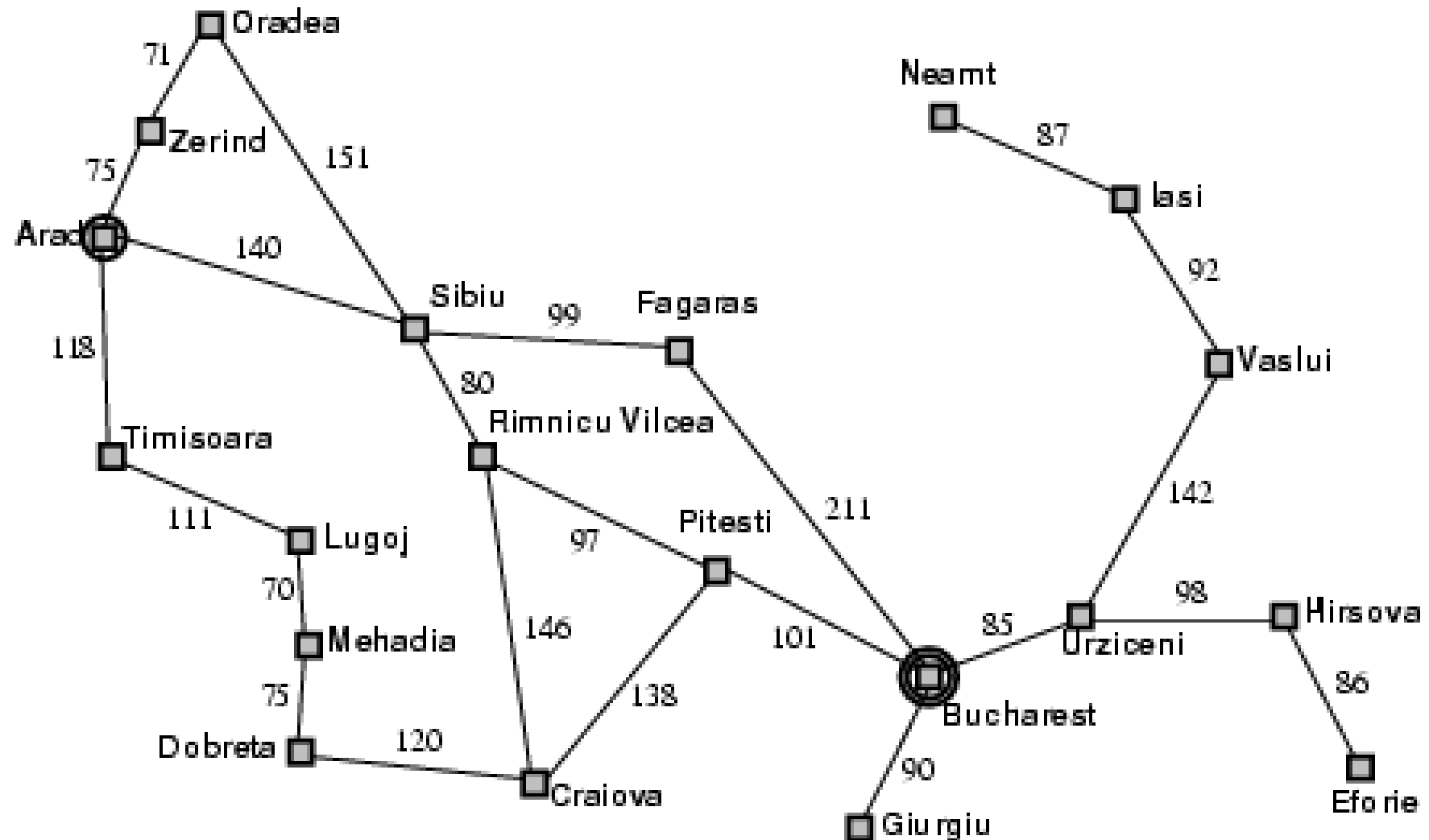
是一种goal-oriented agent

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  static: seq, an action sequence, initially empty
           state, some description of the current world state
           goal, a goal, initially null
           problem, a problem formulation

  state  $\leftarrow$  UPDATE-STATE(state, percept)
  if seq is empty then do
    goal  $\leftarrow$  FORMULATE-GOAL(state)
    problem  $\leftarrow$  FORMULATE-PROBLEM(state, goal)
    seq  $\leftarrow$  SEARCH(problem)
  action  $\leftarrow$  FIRST(seq)
  seq  $\leftarrow$  REST(seq)
  return action
```

Note: this is **offline** problem solving; solution executed “eyes closed.”
Online problem solving involves acting without complete knowledge.

Example: Romania



Example: Romania

- On holiday in Romania; currently in Arad.
- Flight leaves tomorrow from Bucharest
-
- Formulate goal:
 - be in Bucharest
 -
- Formulate problem:
 - **states**: various cities
 - **actions**: drive between cities
 -
- Find solution:
 - sequence of cities, e.g., Arad, Sibiu, Fagaras, Bucharest
 -

Single-state problem formulation

A **problem** is defined by four items:

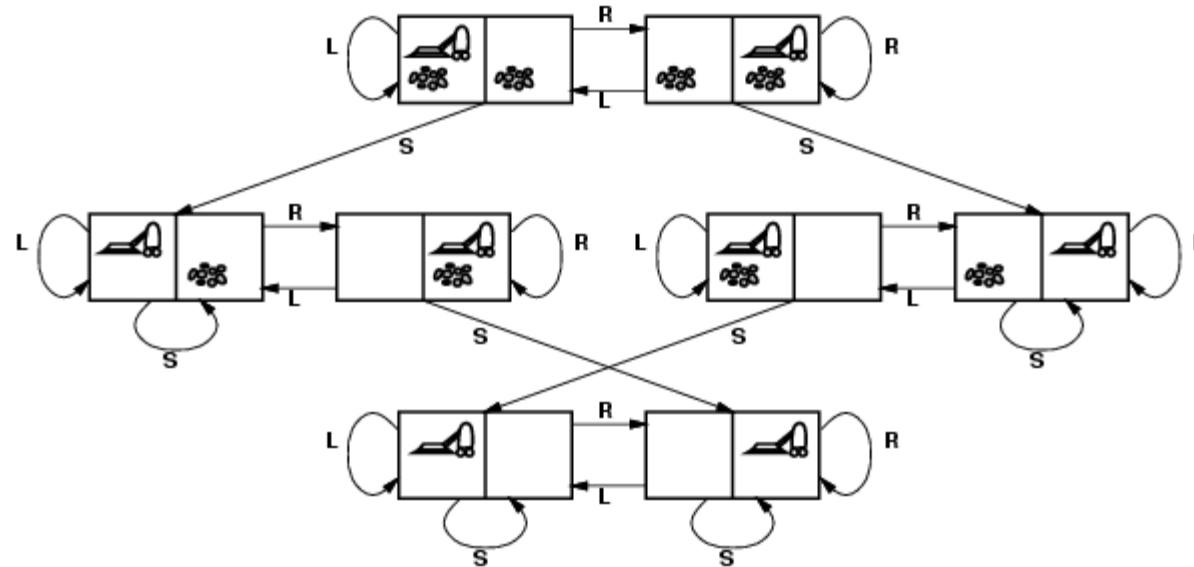
1. **initial state** e.g., "at Arad"
2. **actions** or **successor function** $S(x)$ = set of action–state pairs
 - e.g., $S(\text{Arad}) = \{ \langle \text{Arad} \rightarrow \text{Zerind}, \text{Zerind} \rangle, \dots \}$
 -
3. **goal test**, can be
 - **explicit**, e.g., $x = \text{"at Bucharest"}$
 - **implicit**, e.g., $\text{Checkmate}(x)$
 -
4. **path cost** (additive)
 - e.g., sum of distances, number of actions executed, etc.
 - $c(x,a,y)$ is the **step cost**, assumed to be ≥ 0
 -

- A **solution** is a sequence of actions leading from the initial state to a goal state

Selecting a state space

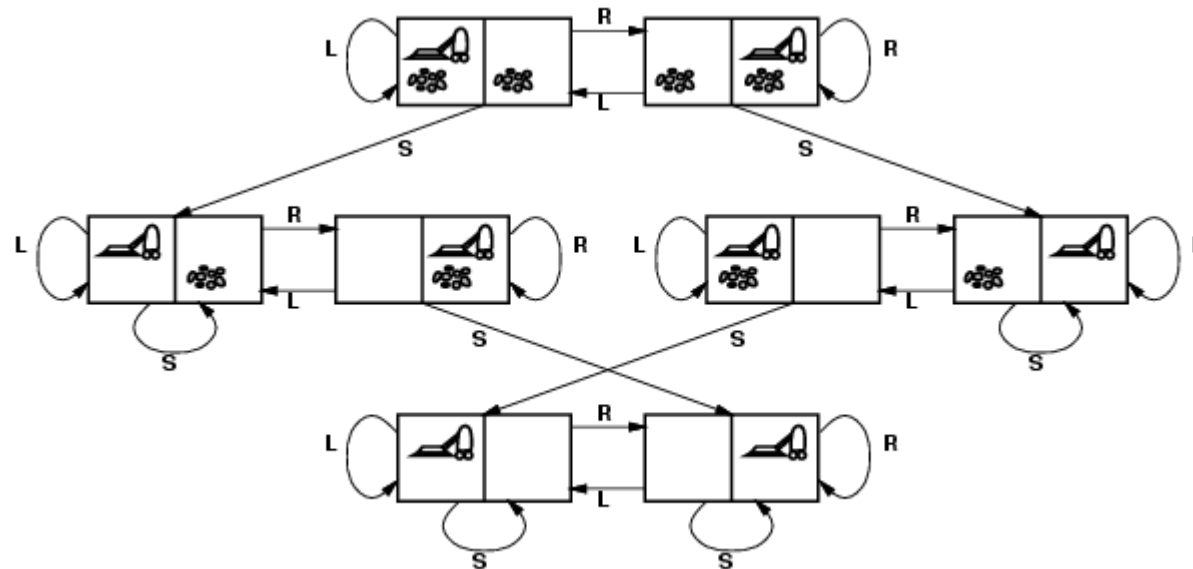
- Real world is absurdly complex
 - state space must be **abstracted** for problem solving
- (Abstract) state = set of real states
-
- (Abstract) action = complex combination of real actions
 - e.g., "Arad → Zerind" represents a complex set of possible routes, detours, rest stops, etc.
- For guaranteed realizability, **any** real state “in Arad” must get to **some** real state "in Zerind"
-
- (Abstract) solution =
 - set of real paths that are solutions in the real world
 -
- Our abstraction should be "easier" to solve than the original problem
-

Vacuum world state space graph



- states?
- actions?
- goal test?
- path cost?
-

Vacuum world state space graph



- states? dirt and robot location
- actions? *Left, Right, Suck, NoOp*
- goal test? no dirt at all locations
- path cost? 1 per action

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- states?
- actions?
- goal test?
- path cost?

Example: The 8-puzzle

7	2	4
5		6
8	3	1

Start State

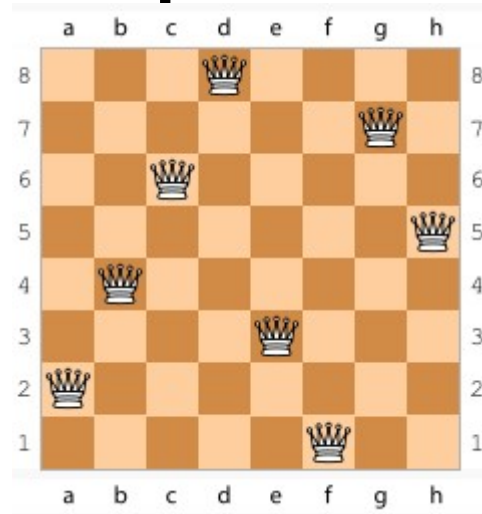
	1	2
3	4	5
6	7	8

Goal State

- states? locations of tiles
- actions? move blank left, right, up, down
- goal test? = goal state (given)
- path cost? 1 per move
-

[Note: optimal solution of n -Puzzle family is NP-hard]

Example: 8皇后问题



问题的2种不同的形式化:

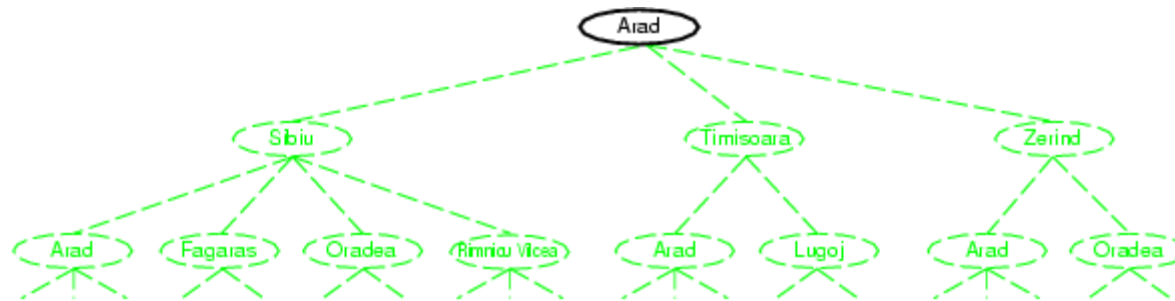
1. 增量形式化: 每次增加一个皇后
2. 完全状态形式化: 八个都在棋盘上, 移动使之不冲突

Tree search algorithms

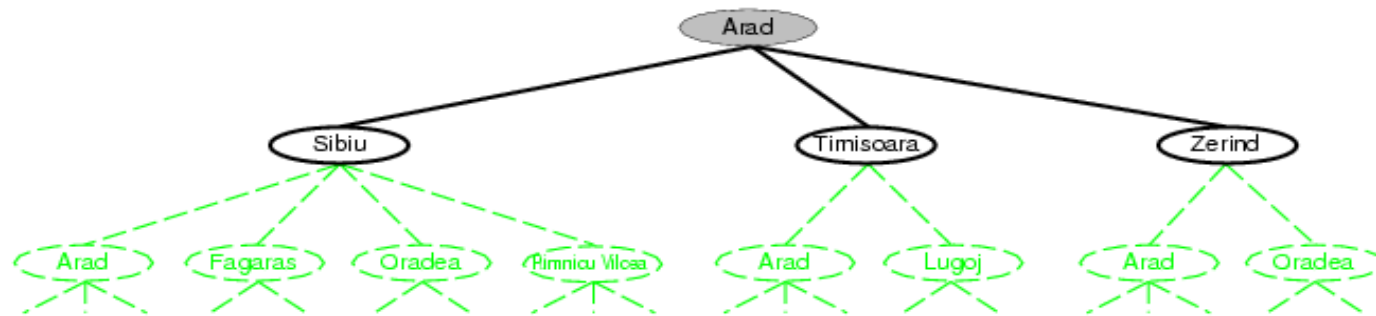
- Basic idea:
 - offline, simulated exploration of state space by generating successors of already-explored states (a.k.a. ~expanding states)

```
function TREE-SEARCH(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
```

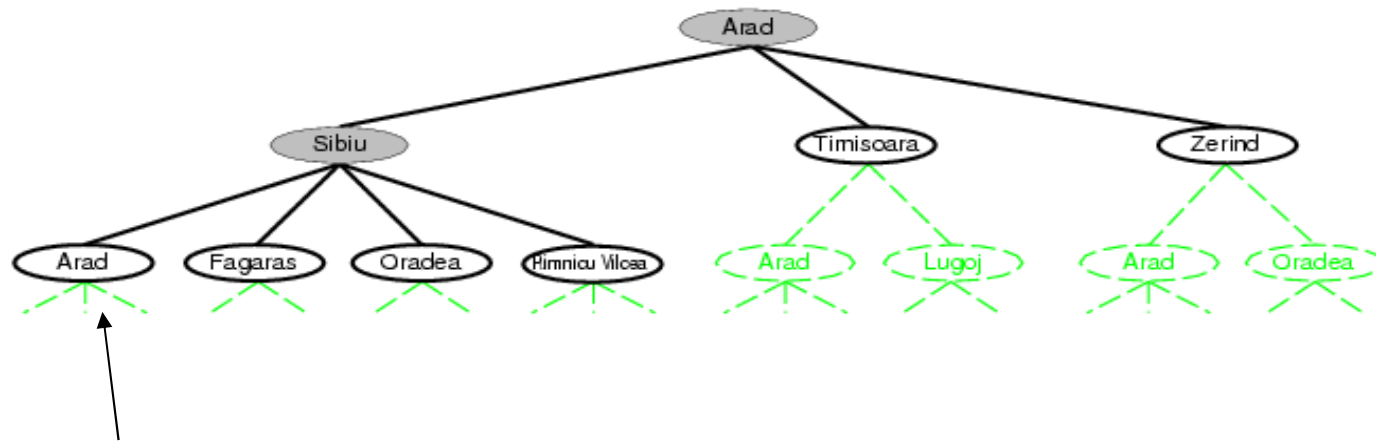
Tree search example



Tree search example



Tree search example



树搜索会产生重复节点

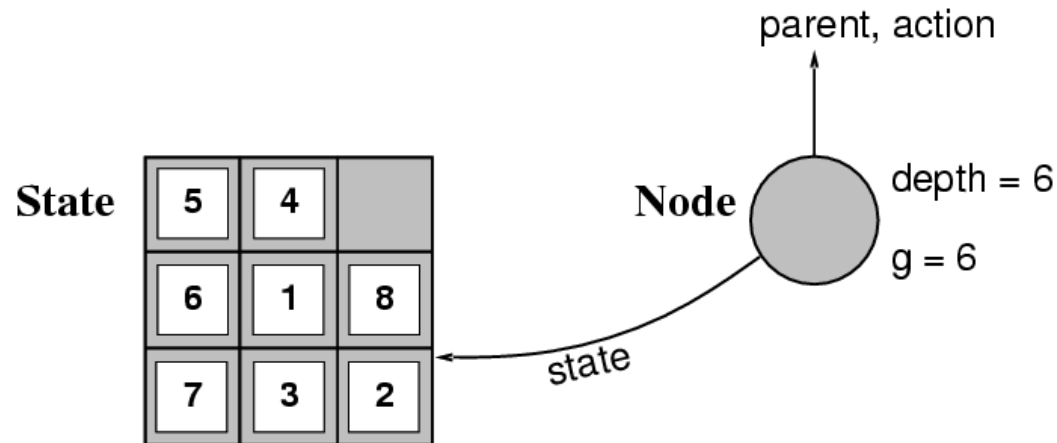
Implementation: general tree search

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure  
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)  
  loop do  
    if fringe is empty then return failure  
    node  $\leftarrow$  REMOVE-FRONT(fringe)  
    if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)  
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function EXPAND(node, problem) returns a set of nodes  
  successors  $\leftarrow$  the empty set  
  for each action, result in SUCCESSOR-FN[problem](STATE[node]) do  
    s  $\leftarrow$  a new NODE  
    PARENT-NODE[s]  $\leftarrow$  node; ACTION[s]  $\leftarrow$  action; STATE[s]  $\leftarrow$  result  
    PATH-COST[s]  $\leftarrow$  PATH-COST[node] + STEP-COST(node, action, s)  
    DEPTH[s]  $\leftarrow$  DEPTH[node] + 1  
    add s to successors  
  return successors
```

Implementation: states vs. nodes

- A **state** is an (abstract representation of) a physical configuration
- A **node** is a data structure constituting part of a search tree includes **state**, **parent node**, **action**, **path cost $g(x)$** , **depth**



- The `Expand` function creates new nodes, filling in the various fields and using the `SuccessorFn` of the problem to create the corresponding states.
-

Search strategies

- A search strategy is defined by picking the **order of node expansion**
- Strategies are evaluated along the following dimensions:
 - **completeness**: does it always find a solution if one exists?
 - **time complexity**: number of nodes generated
 - **space complexity**: maximum number of nodes in memory
 - **optimality**: does it always find a least-cost solution?
 -
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)
 -

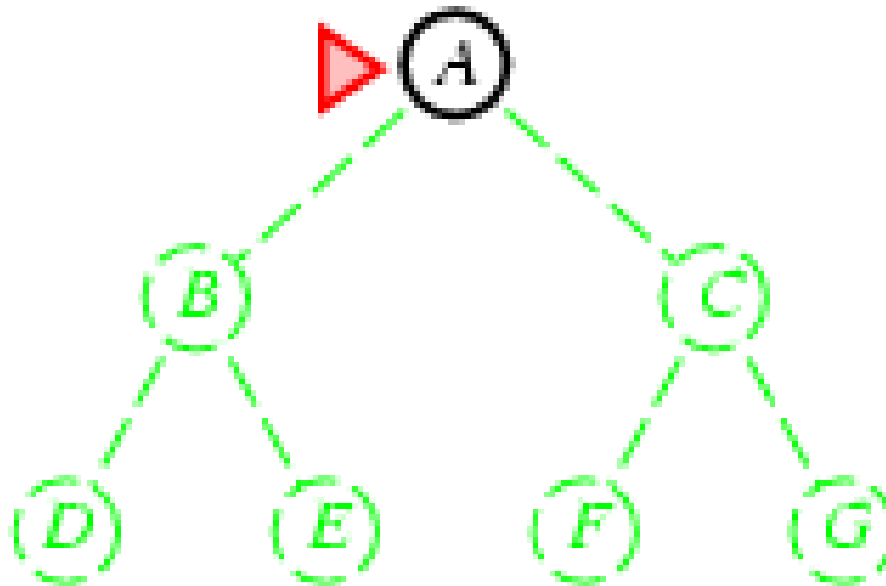
Uninformed search strategies

- **Uninformed** search strategies use only the information available in the problem definition
 - Breadth-first search
 - Uniform-cost search (应该叫min-cost search)
 - Depth-first search
 - Depth-limited search
 - Iterative deepening search
 -

Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *fringe* is a FIFO queue, i.e., new successors go at end

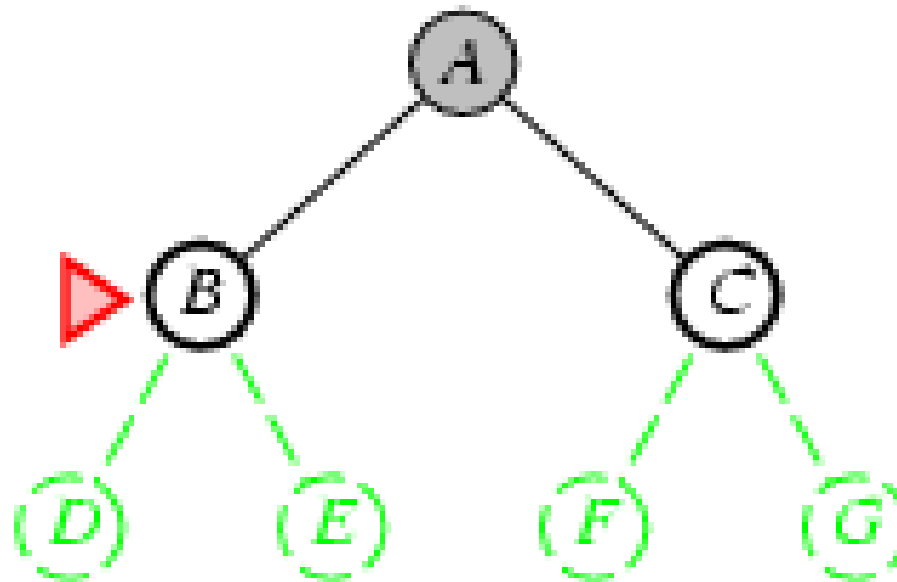
–



Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *fringe* is a FIFO queue, i.e., new successors go at end

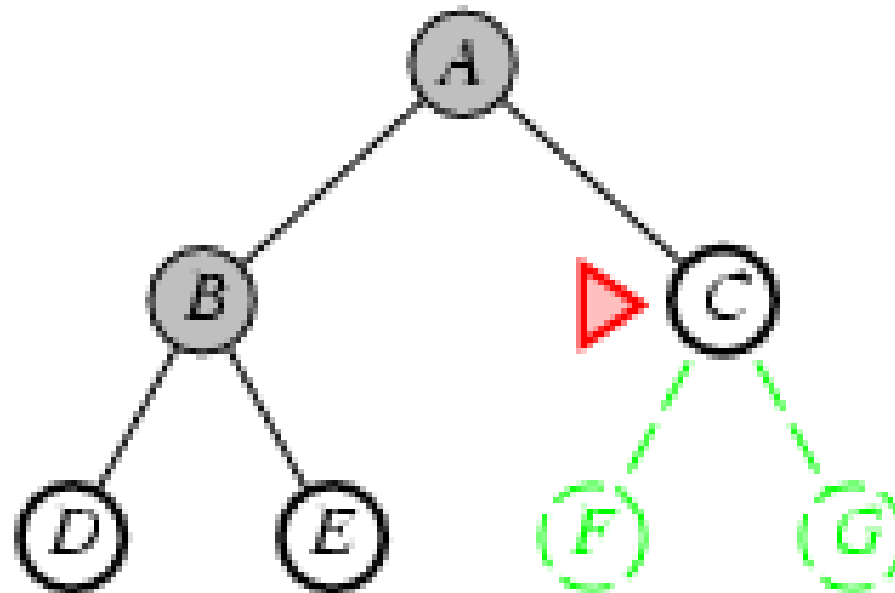
–



Breadth-first search

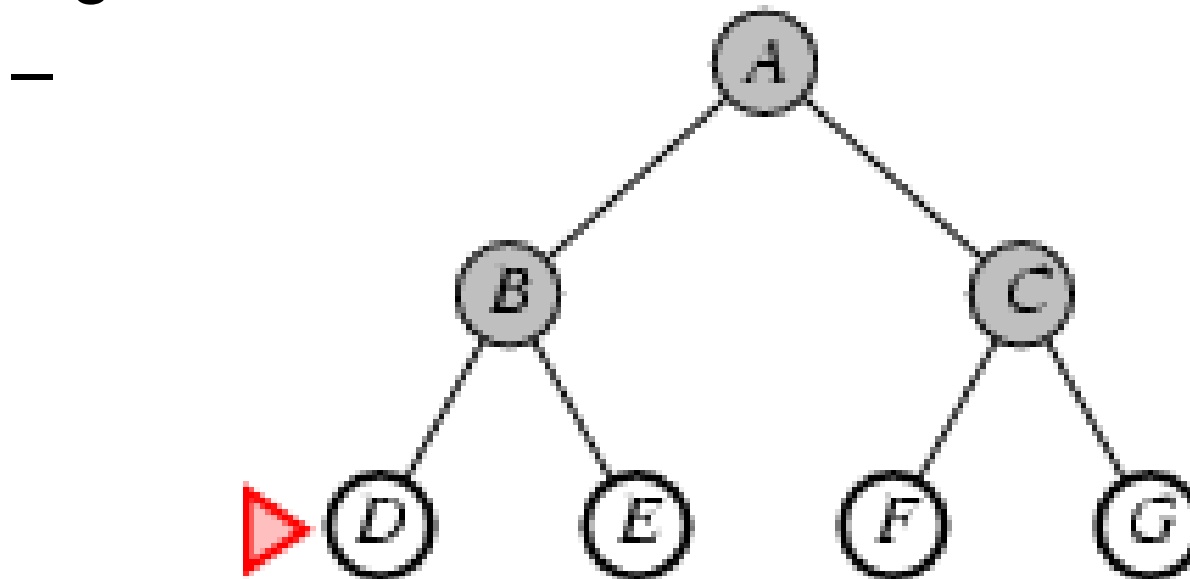
- Expand shallowest unexpanded node
- **Implementation:**
 - *fringe* is a FIFO queue, i.e., new successors go at end

–



Breadth-first search

- Expand shallowest unexpanded node
- **Implementation:**
 - *fringe* is a FIFO queue, i.e., new successors go at end



Properties of breadth-first search

- Complete? Yes (if b is finite)
-
- Time? $1+b+b^2+b^3+\dots +b^d + b(b^d-1) = O(b^{d+1})$
-
- Space? $O(b^{d+1})$ (keeps every node in memory)
-
- Optimal? Yes (if cost = 1 per step)
-
- **Space** is the bigger problem (more than time)
-

Uniform-cost search

Expand least-cost unexpanded node

Implementation:

fringe = queue ordered by path cost, lowest first

Equivalent to breadth-first if step costs all equal

Complete?? Yes, if step cost $\geq \epsilon$

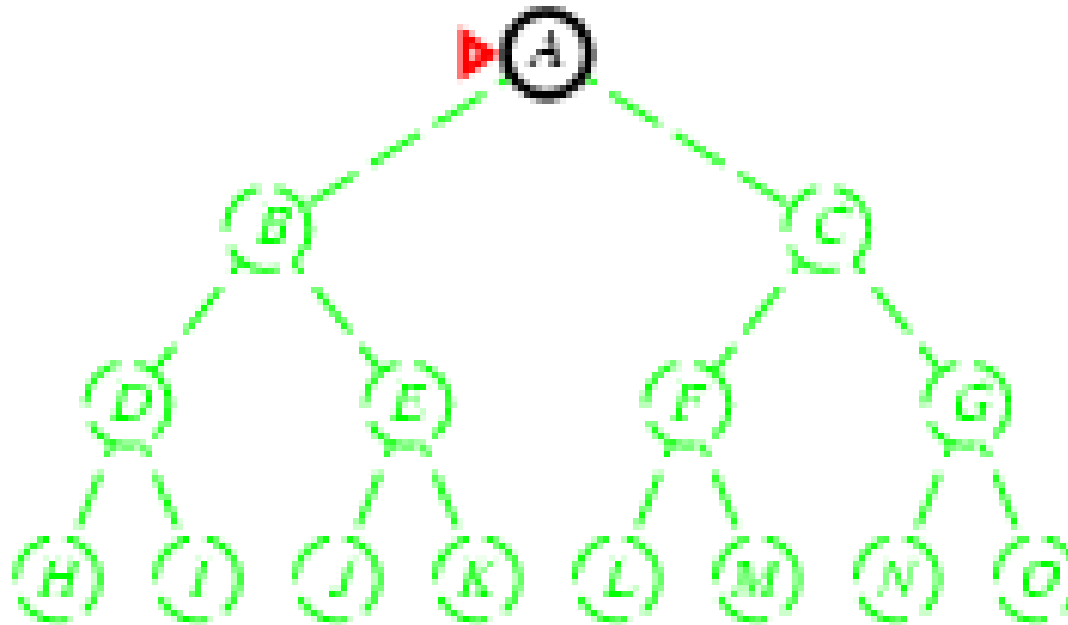
Time?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$
where C^* is the cost of the optimal solution

Space?? # of nodes with $g \leq$ cost of optimal solution, $O(b^{\lceil C^*/\epsilon \rceil})$

Optimal?? Yes—nodes expanded in increasing order of $g(n)$

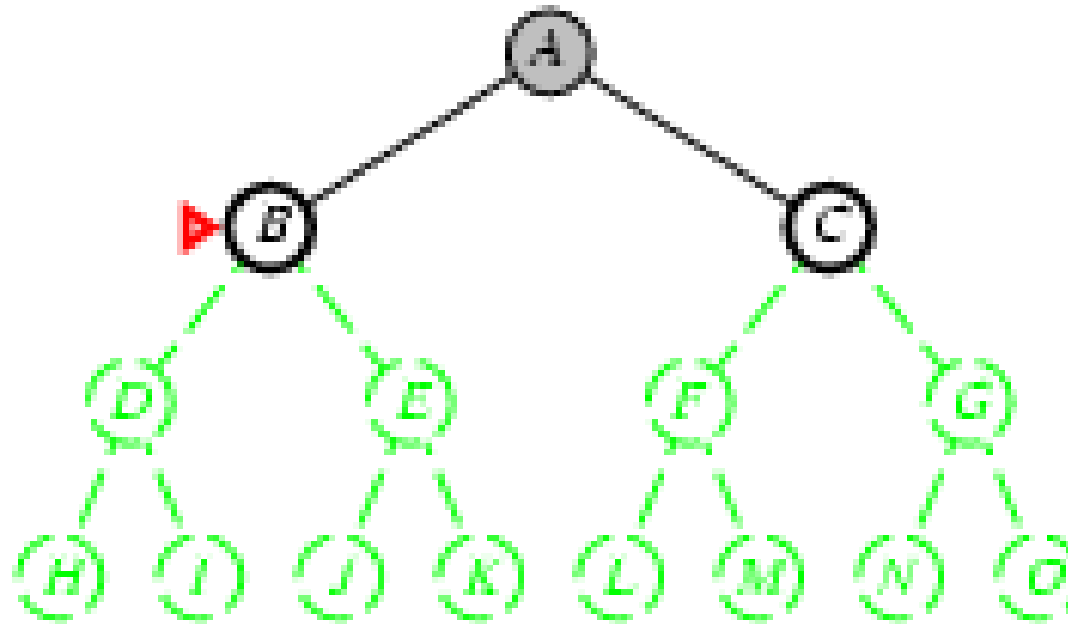
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



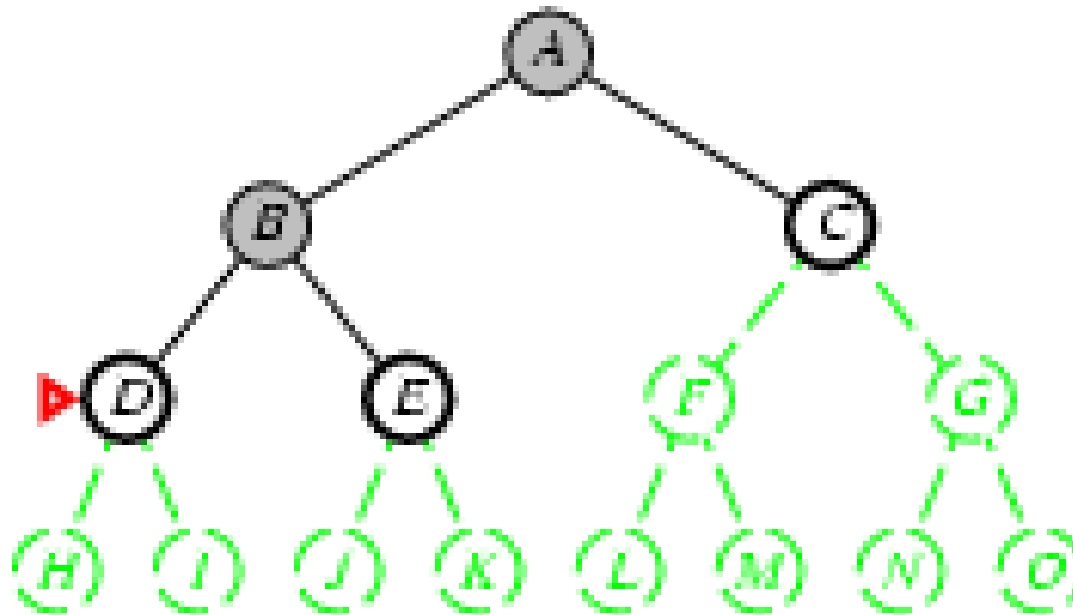
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



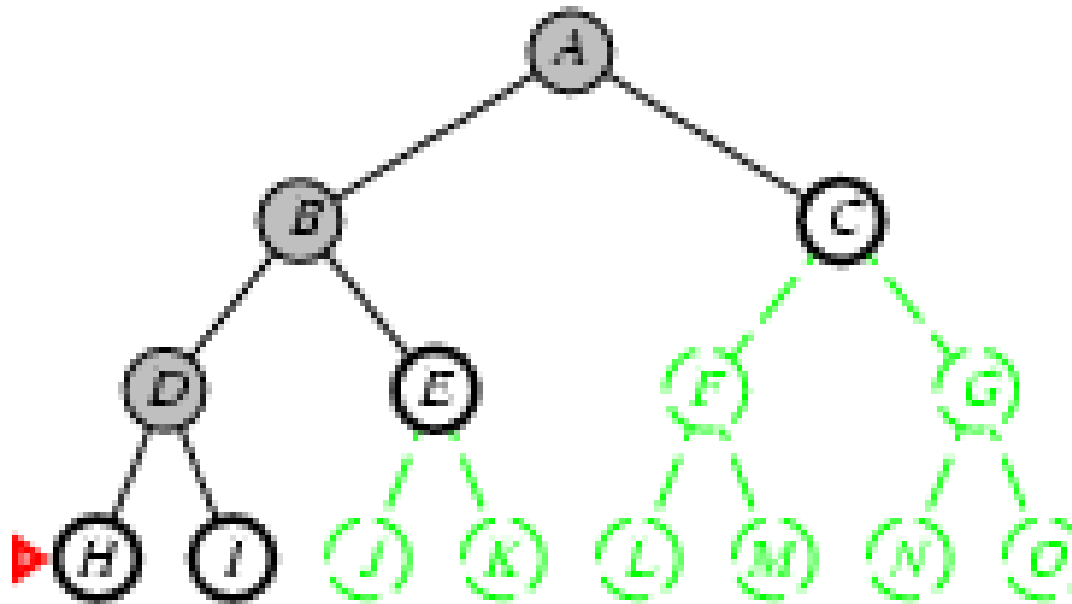
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



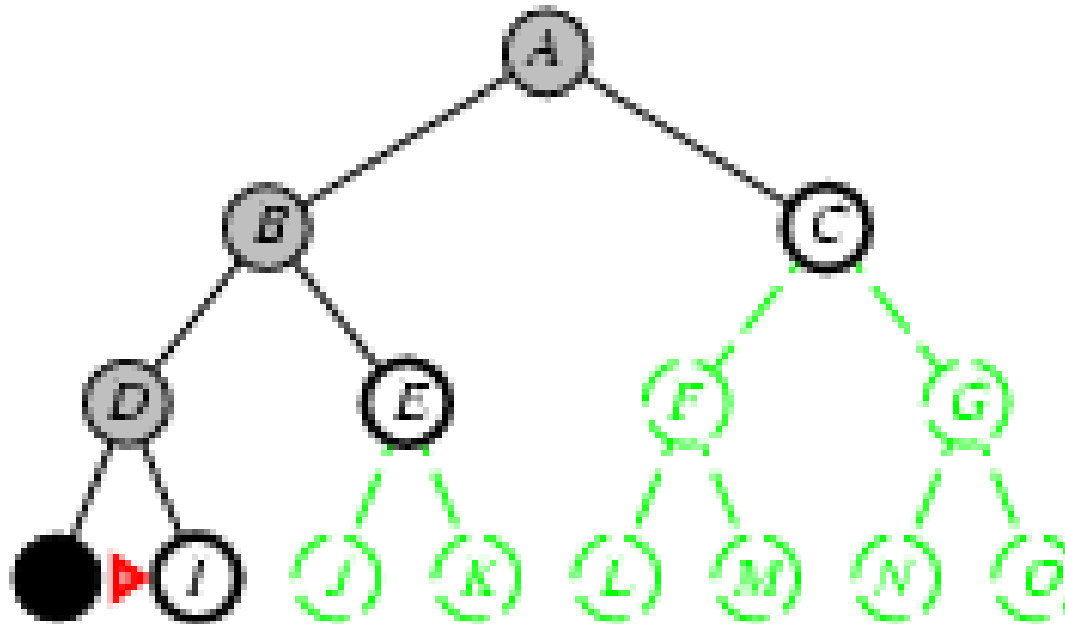
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



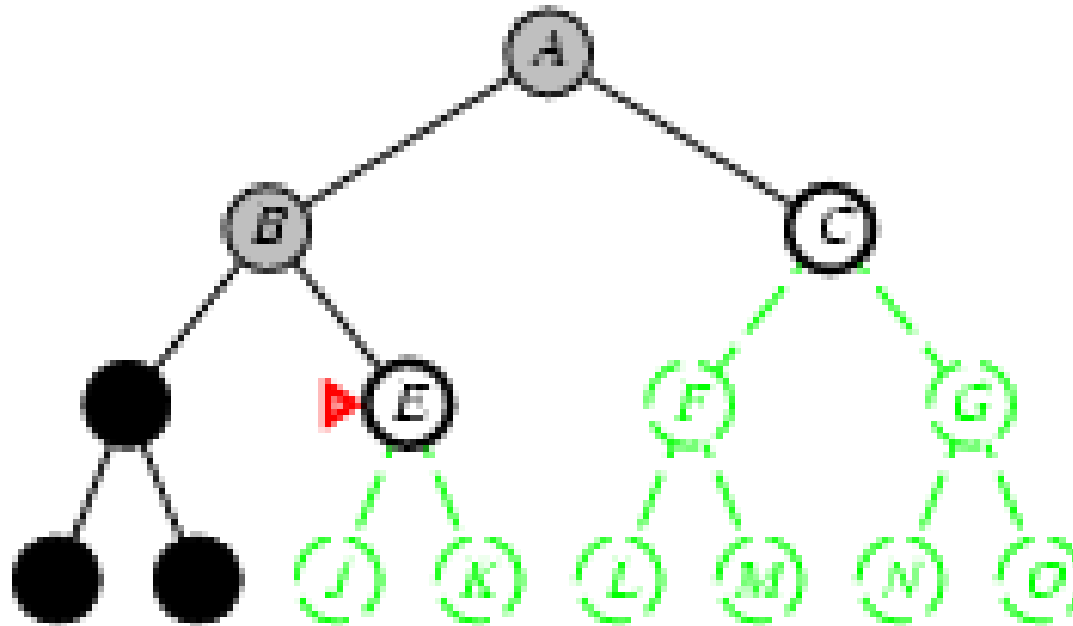
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



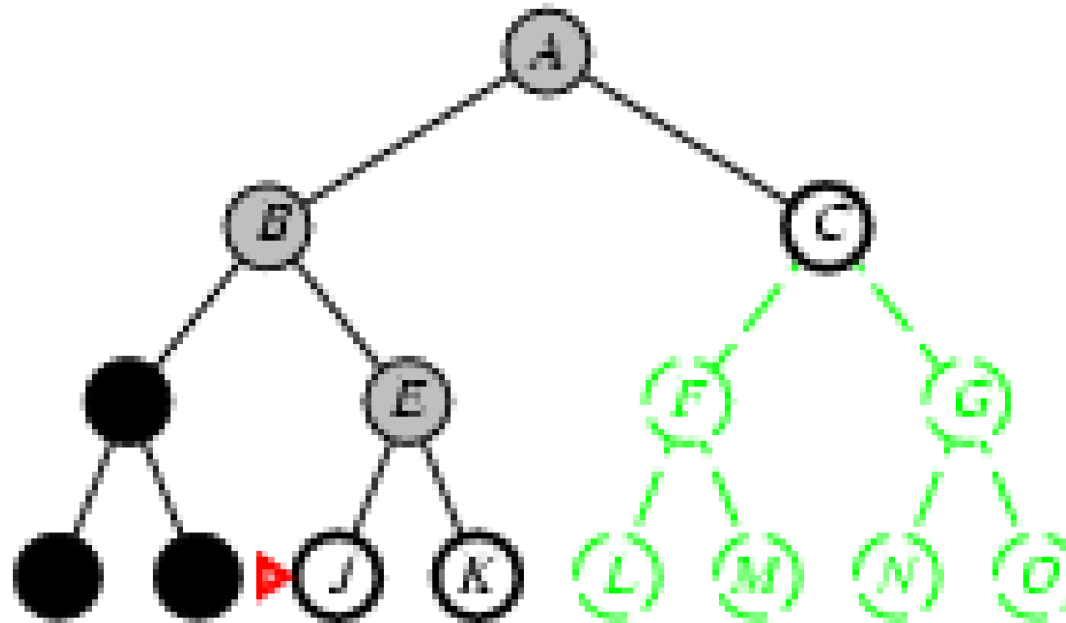
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



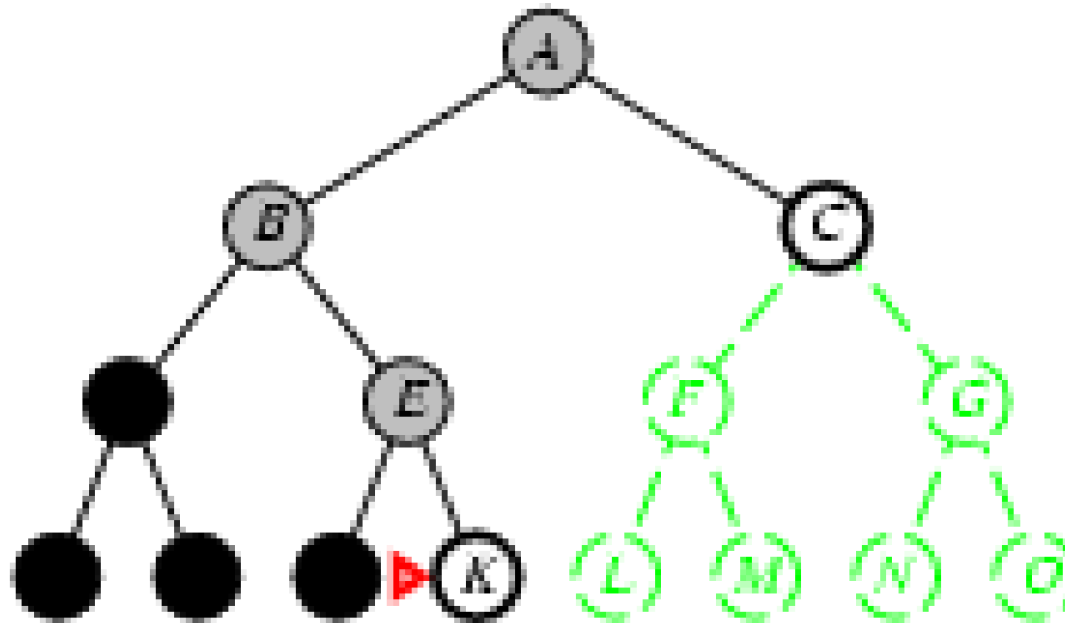
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



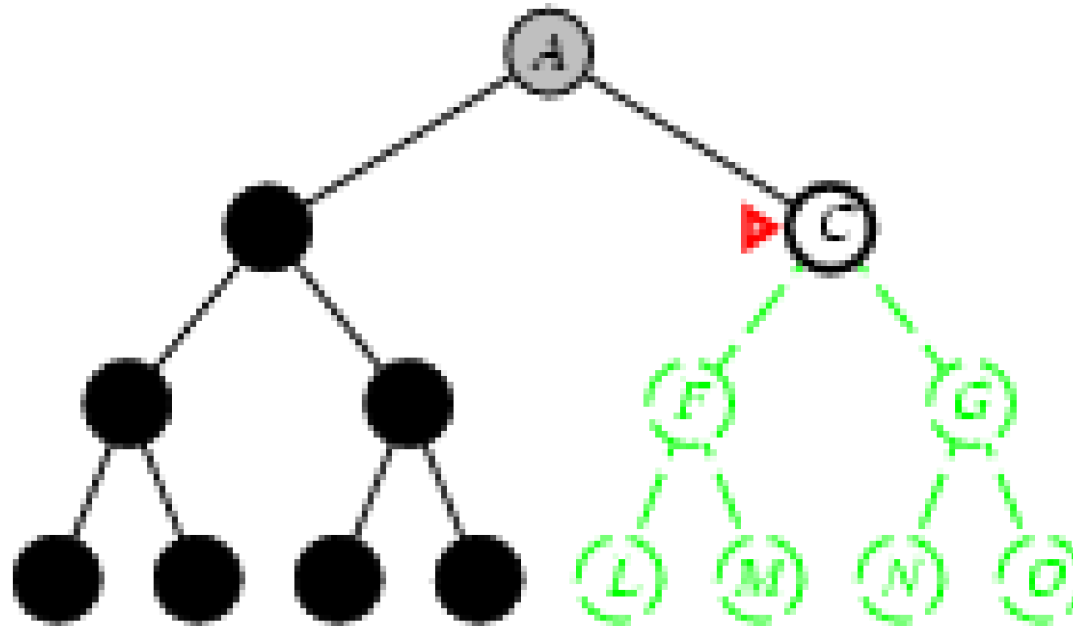
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



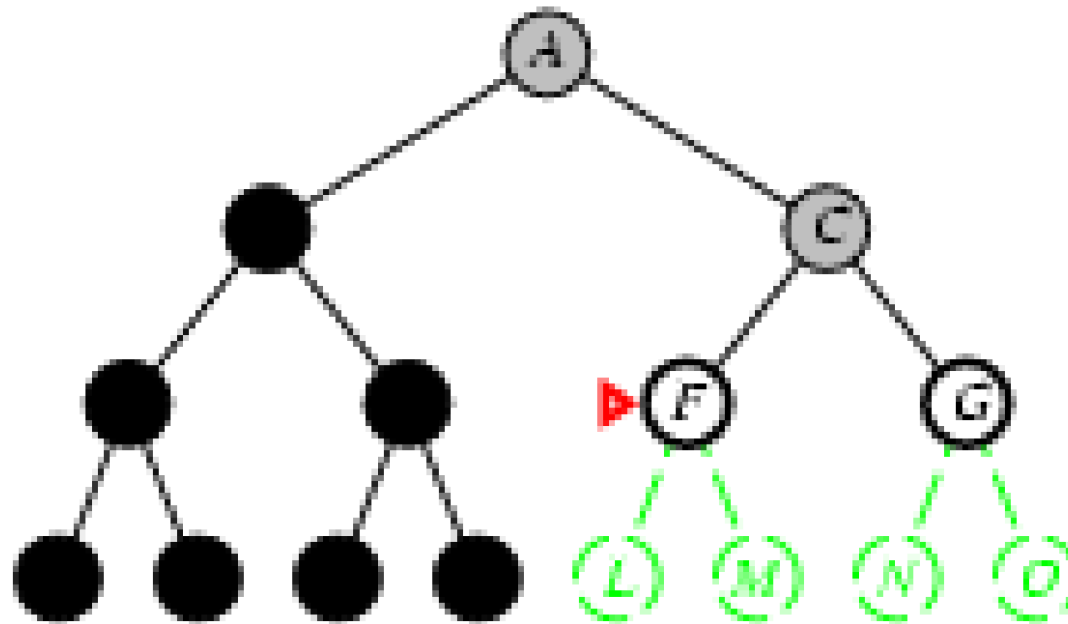
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



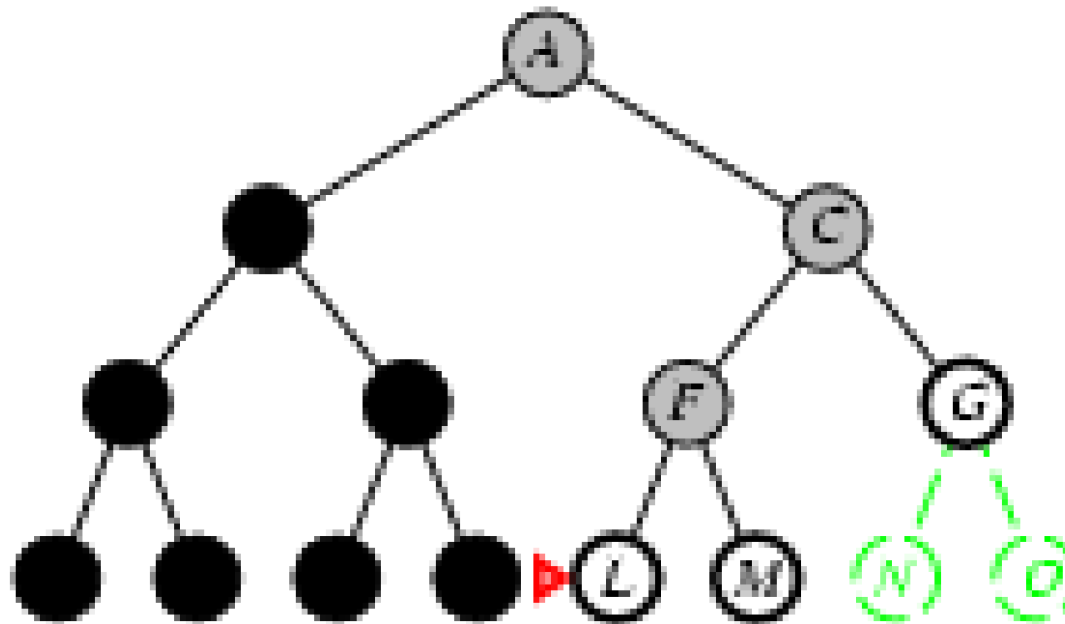
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



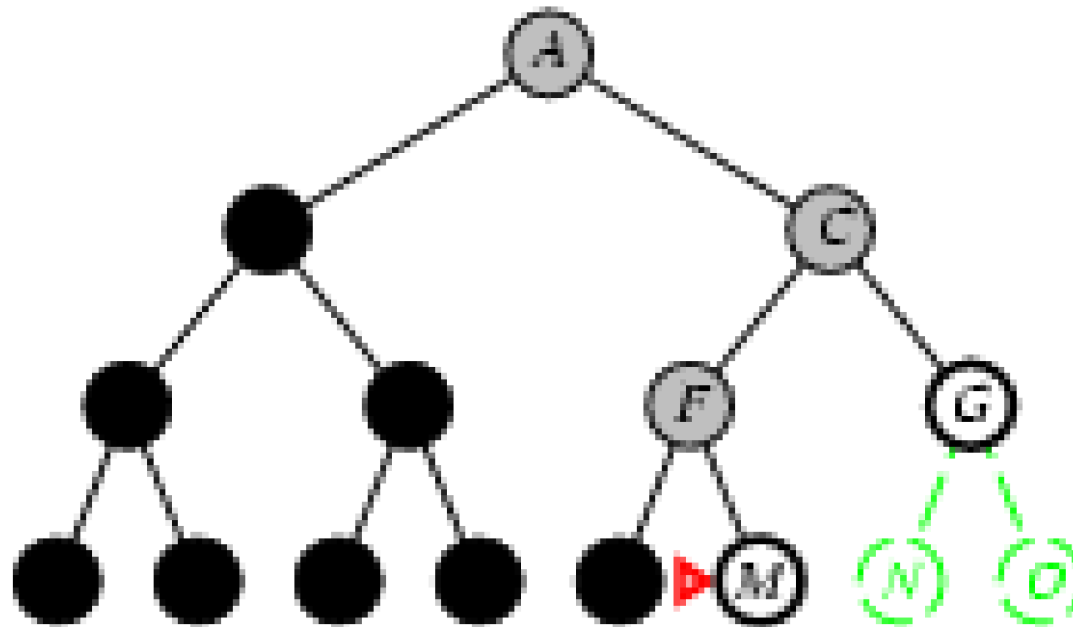
Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front



Depth-first search

- Expand deepest unexpanded node
- **Implementation:**
 - *fringe* = LIFO queue, i.e., put successors at front
 -



Properties of depth-first search

- Complete? No: fails in infinite-depth spaces, spaces with loops
 - Modify to avoid repeated states along path
 - → complete in finite spaces
 -
- Time? $O(b^m)$: terrible if m is much larger than d
 - but if solutions are dense, may be much faster than breadth-first
- Space? $O(bm)$, i.e., linear space!
- Optimal? No
-

Depth-limited search

= depth-first search with depth limit l ,
i.e., nodes at depth l have no successors

- ```
function DEPTH-LIMITED-SEARCH(problem, limit) returns soln/fail/cutoff
 RECURSIVE-DLS(MAKE-NODE(INITIAL-STATE[problem]), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns soln/fail/cutoff
 cutoff-occurred? $\leftarrow$ false
 if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
 else if DEPTH[node] = limit then return cutoff
 else for each successor in EXPAND(node, problem) do
 result $\leftarrow$ RECURSIVE-DLS(successor, problem, limit)
 if result = cutoff then cutoff-occurred? $\leftarrow$ true
 else if result $\neq$ failure then return result
 if cutoff-occurred? then return cutoff else return failure
```



# Iterative deepening search

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or fail-
ure
 inputs: problem, a problem
 for depth $\leftarrow$ 0 to ∞ do
 result $\leftarrow$ DEPTH-LIMITED-SEARCH(problem, depth)
 if result $\neq$ cutoff then return result
```

# Iterative deepening search / =0

Limit = 0



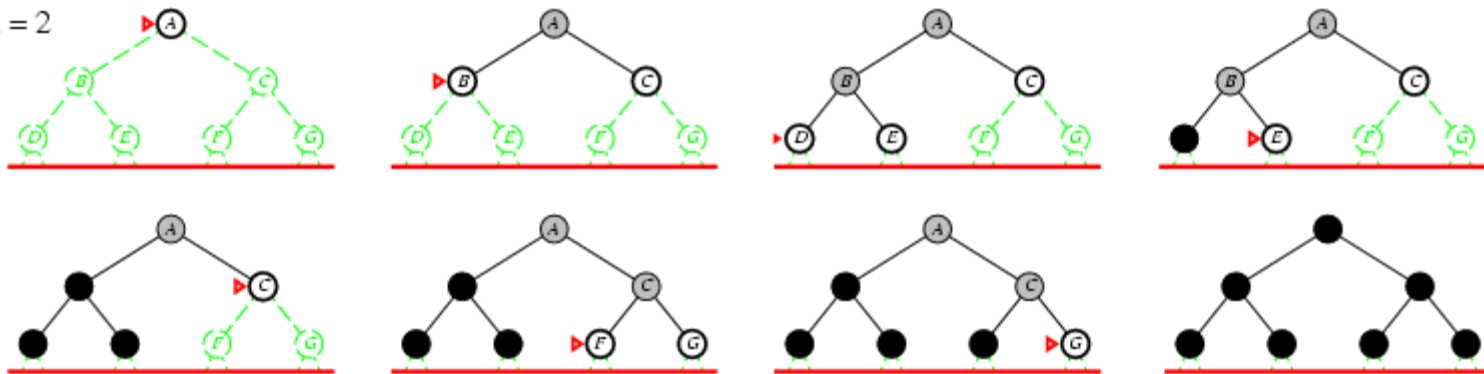
# Iterative deepening search / =1

Limit = 1



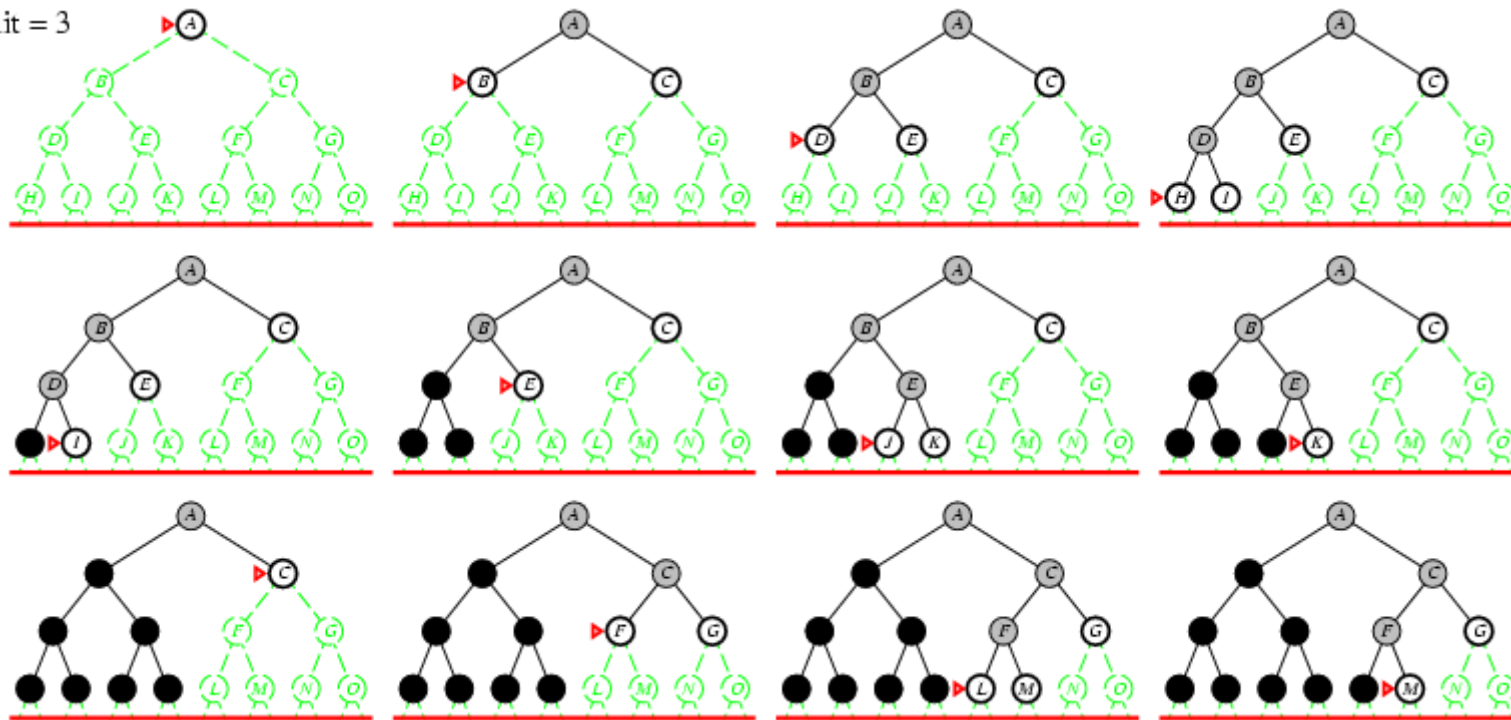
# Iterative deepening search / =2

Limit = 2



# Iterative deepening search / =3

Limit = 3



# Iterative deepening search

- Number of nodes generated in a depth-limited search to depth  $d$  with branching factor  $b$ :

$$N_{DLS} = b^0 + b^1 + b^2 + \dots + b^{d-2} + b^{d-1} + b^d$$

- Number of nodes generated in an iterative deepening search to depth  $d$  with branching factor  $b$ :

$$N_{IDS} = (d+1)b^0 + d b^1 + (d-1)b^2 + \dots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

- For  $b = 10$ ,  $d = 5$ ,

- 

- $N_{DLS} = 1 + 10 + 100 + 1,000 + 10,000 + 100,000 = 111,111$

- 

- $N_{IDS} = 6 + 50 + 400 + 3,000 + 20,000 + 100,000 = 123,456$

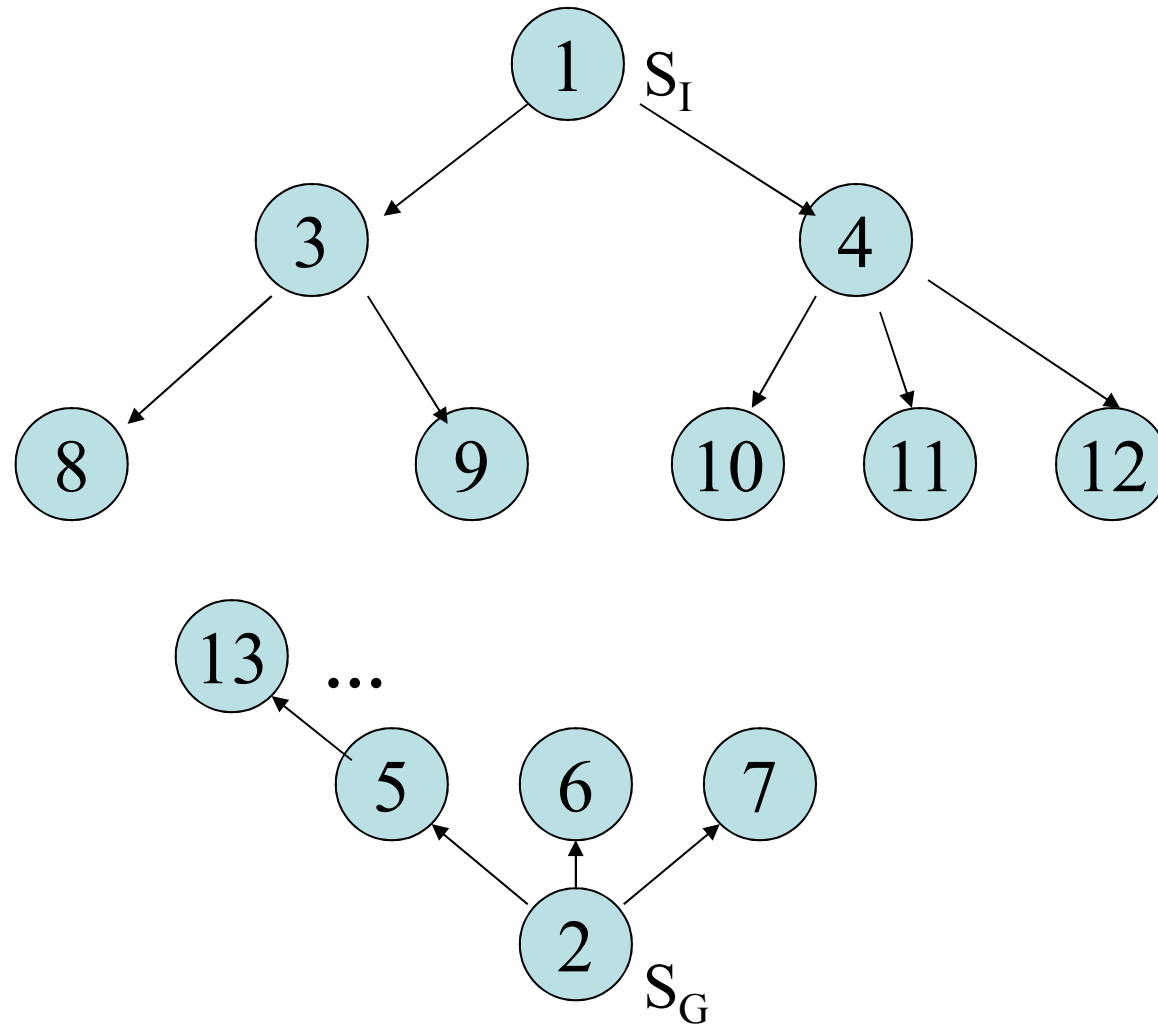
- 

- Overhead =  $(123,456 - 111,111)/111,111 = 11\%$

# Properties of iterative deepening search

- Complete? Yes
- 
- Time?  $(d+1)b^0 + d b^1 + (d-1)b^2 + \dots + b^d = O(b^d)$
- 
- Space?  $O(bd)$
- 
- Optimal? Yes, if step cost = 1

# Bi-Directional Breadth-First Search



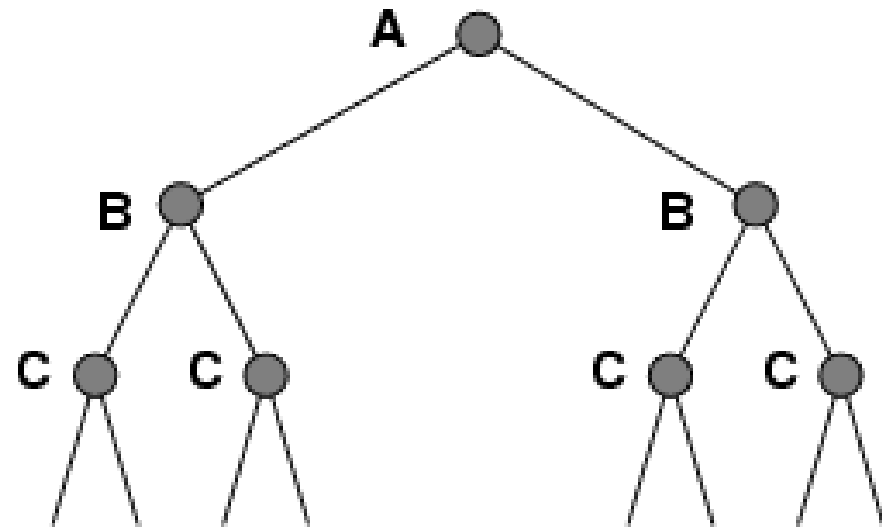
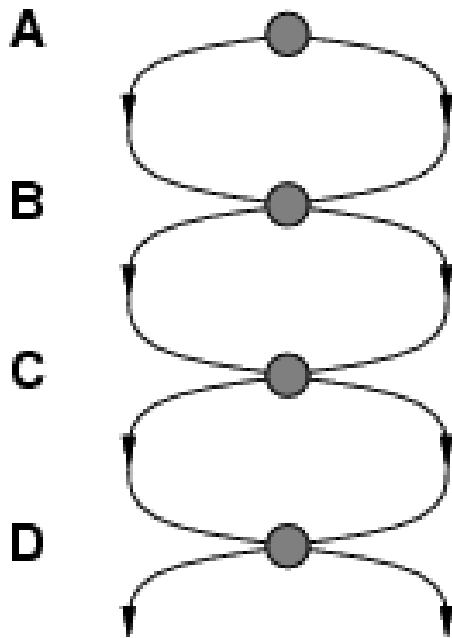


# Summary of algorithms

| Criterion | Breadth-First | Uniform-Cost                        | Depth-First | Depth-Limited | Iterative Deepening |
|-----------|---------------|-------------------------------------|-------------|---------------|---------------------|
| Complete? | Yes           | Yes                                 | No          | No            | Yes                 |
| Time      | $O(b^{d+1})$  | $O(b^{\lceil C^*/\epsilon \rceil})$ | $O(b^m)$    | $O(b^l)$      | $O(b^d)$            |
| Space     | $O(b^{d+1})$  | $O(b^{\lceil C^*/\epsilon \rceil})$ | $O(bm)$     | $O(bl)$       | $O(bd)$             |
| Optimal?  | Yes           | Yes                                 | No          | No            | Yes                 |

# Repeated states

- Failure to detect repeated states can turn a linear problem into an exponential one!



# Graph search

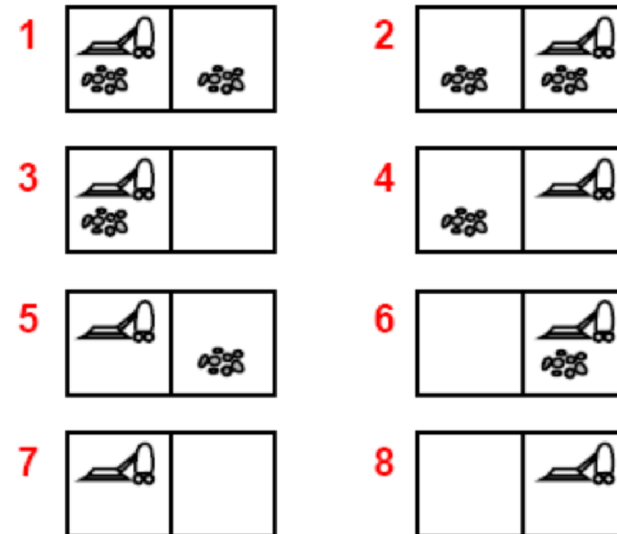
```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
 closed ← an empty set
 fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
 loop do
 if fringe is empty then return failure
 node ← REMOVE-FRONT(fringe)
 if GOAL-TEST[problem](STATE[node]) then return SOLUTION(node)
 if STATE[node] is not in closed then
 add STATE[node] to closed
 fringe ← INSERTALL(EXPAND(node, problem), fringe)
```

# Problem types

- Deterministic, fully observable → single-state problem
  - Agent knows exactly which state it will be in; solution is a sequence
- Non-observable → sensorless problem (conformant problem)
  - Agent may have no idea where it is; solution is a sequence
  - belief states
- Nondeterministic and/or partially observable → contingency problem
  - percepts provide new information about current state
  - solution is a contingent (随机而动) plan or a policy
  - often interleave search, execution
- Unknown state space → exploration problem (“online”)

## Example: vacuum world

Single-state, start in #5. [Solution??](#)



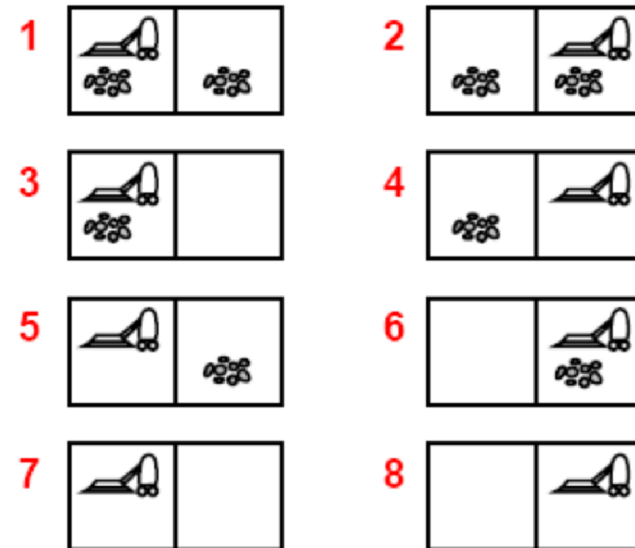
## Example: vacuum world

Single-state, start in #5. Solution??

[*Right, Suck*]

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}

e.g., *Right* goes to {2, 4, 6, 8}. Solution??



## Example: vacuum world

Single-state, start in #5. Solution??

*[Right, Suck]*

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}  
e.g., *Right* goes to {2, 4, 6, 8}. Solution??

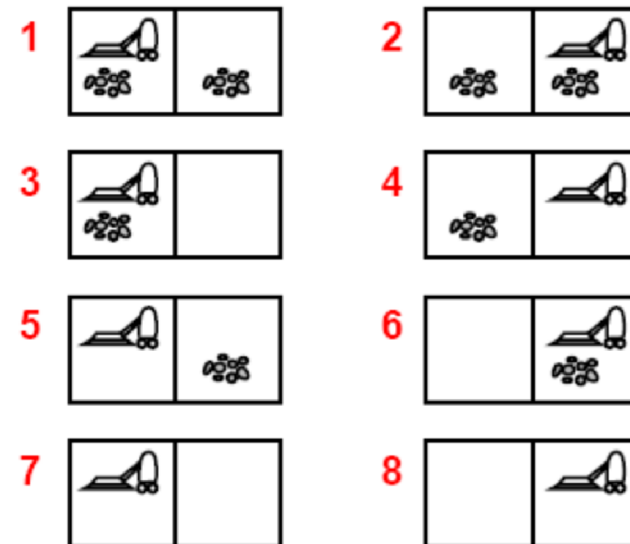
*[Right, Suck, Left, Suck]*

Contingency, start in #5

Murphy's Law: *Suck* can dirty a clean carpet

Local sensing: dirt, location only.

Solution??



## Example: vacuum world

Single-state, start in #5. Solution??

[*Right, Suck*]

Conformant, start in {1, 2, 3, 4, 5, 6, 7, 8}

e.g., *Right* goes to {2, 4, 6, 8}. Solution??

[*Right, Suck, Left, Suck*]

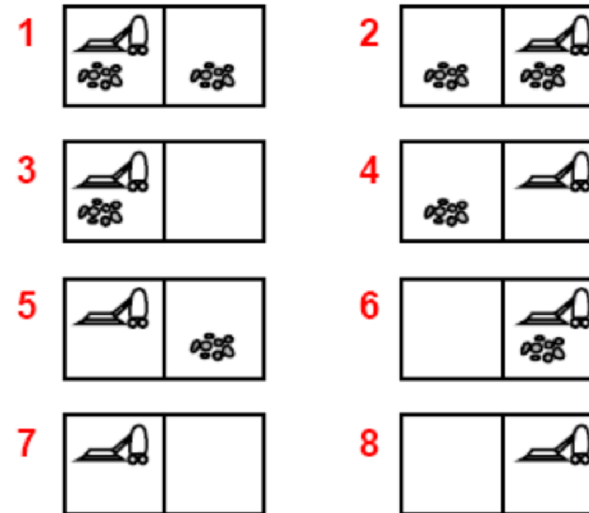
Contingency, start in #5

Murphy's Law: *Suck* can dirty a clean carpet

Local sensing: dirt, location only.

Solution??

[*Right, if dirt then Suck*]





# Summary

- Problem formulation usually requires abstracting away real-world details to define a state space that can feasibly be explored
- 
- Variety of uninformed search strategies
- 
- Iterative deepening search uses only linear space and not much more time than other uninformed algorithms
-

# 作业

- 考虑一个状态空间，其初始编号为1，状态 $n$ 的后继函数返回编号为 $2n$ 与 $2n+1$ 的两个状态。
  - a. 画出状态1到15的部分状态空间图。
  - b. 假设目标状态是11，列出用以下算法访问节点的顺序：广度优先搜索、深度限制为3的有限深度搜索和迭代加深搜索
  - c. 双向搜索是否适合该问题？如适合，详述其工作原理
  - d. 在双向搜索中每个方向上的分叉因子是什么？
  - e. 对(c)的回答是否能提出该问题的另一种形式化，使得你可以几乎不用搜索来求解从状态1到达目标状态的问题？

# 作业

- 有名的传教士和野人问题（Amarel, 1968）：三个传教士和三个野人需从河的一边用一条只能载最多2个人的船渡到对岸。但是需保证任意时刻野人的个数不大于传教士个数（否则传教士会被野人吃掉）。
  - a. 精确地形式化该问题，画出完全的状态空间。
  - b. 用一个合适的搜索算法求该问题的最优解。检查重复状态是个好主意吗？
  - c. 既然该问题的状态空间如此简单，为何人们求解她却显得困难？

# 作业

- 编写一个程序，当输入两个网页的URL后，试着找到从一个网页到另一个网页的链接路径。什么样的搜索策略比较合适？双向搜索是好主意吗？能用搜索引擎实现一个前驱函数？

# 课外阅读

- 研究一下“数独”游戏：  
<http://en.wikipedia.org/wiki/Sudoku>
- 阅读论文：
- Bruce Abramson, [Moti Yung](#): Divide and Conquer under Global Constraints: A Solution to the N-Queens Problem. [J. Parallel Distrib. Comput.](#) **6**(3): 649-662 (1989)